

CYBERGUARD

Master Developer Documentation

BPUT Hackathon · Problem Statement 9 · v3.0

§ 01 What We Build

CyberGuard is an agentic AI cybersecurity platform built on AgentScope 1.0.21. The system detects, classifies, scores, and explains cyber threats in near real time using an event-driven multi-agent pipeline. Every component is replaceable behind an interface. No concrete dependency lives inside any agent or detector.

Required Demo Scenarios

SCENARIO	NAME	SCOPE
01 — Primary	Phishing Detection	Email, URL, SMS, QR code phishing via NLP classification and domain analysis
02 — Primary	Deepfake Detection	Image, video frame, and audio manipulation detection via computer vision
03 — Primary	Log Anomaly Detection	Auth log, system log, and behavioural anomaly via unsupervised ML

Warning: Voice Cloning is not in primary scope. Build only if all three primary scenarios are demo-ready and 48 hours remain. No half-integrated features at demo time.

Pipeline Output Per Event

Every event entering the system must produce all six outputs before reaching the dashboard. Partial pipelines are not acceptable.

Detection → Classification → Risk Score → LLM Explanation → Alert → Response Action

§ 02 Team Ownership

Every module has exactly one owner. Ownership means writing, testing, and being accountable for that module at demo time. Cross-module changes require the module owner's review before merge.

DEVELOPER	ROLE	OWNED MODULES	IMPLEMENTS INTERFACE
Ommkar	Architect + Infra + Docs	core/, orchestrator.py, response/, db/, MongoDB schema	All base interfaces
Dev 1	Phishing ML	agents/phishing/	BaseDetector, BaseMLModel, BaseCyberAgent
Dev 2	Deepfake ML	agents/deepfake/	BaseDetector, BaseMLModel, BaseCyberAgent
Dev 3	Log Analysis ML	agents/log_analysis/, agents/api_abuse/	BaseDetector, BaseMLModel, BaseCyberAgent
Dev 4	FastAPI + Ingestion	api/, ingestion/, scoring/	BaseScorer, ThreatEvent schema
Dev 5	Frontend Dashboard	frontend/	WebSocket message types

Critical: Two ownership gaps — assign before Day 2: 1. WebSocket message schema — Dev 4 and Dev 5 must agree on the JSON contract and update Section 14 of this doc. 2. Dataset preprocessing pipeline — who downloads, cleans, and serves demo datasets to each ML agent?

§ 03 Tech Stack

LAYER	TECHNOLOGY	VERSION	PURPOSE
Agent Framework	AgentScope	1.0.21	Event-driven multi-agent orchestration
Backend	Python + FastAPI	3.11 / 0.111+	REST API, WebSocket server, file ingestion
Frontend	Next.js + TypeScript	14 / 5	Unified threat monitoring dashboard
Styling	TailwindCSS	3.4+	Component styling
Database	MongoDB (Motor async)	7.0	Threat events, sessions, escalations
Event Bus	Redis Streams	7.2	Async inter-agent communication — persistent
Auth	JWT (python-jose)	3.3.0	API and WebSocket authentication
LLM Layer	Model-agnostic	—	OpenAI / Groq / Anthropic — swappable via interface
ML Models	Model-agnostic	—	HuggingFace / ONNX / scikit-learn — swappable via interface
Validation	Pydantic v2	2.0+	All event schemas and API models
Async Runtime	asyncio + uvicorn	—	Full async I/O throughout backend

§ 04 System Architecture

Data Flow

Every input — whether a file upload or a simulated live stream — follows this single pipeline. No agent calls another agent directly. All communication is through Redis Streams.

Input → Ingestion Layer → Redis: raw.input → OrchestratorAgent → Domain Agent → Redis: threat.detected → ScoringAgent → ResponseAgent (LLM) → MongoDB → WebSocket → Dashboard

Agent Map

AGENT	SUBSCRIBES TO	EMITS TO	OWNER
OrchestratorAgent	cyberguard:raw.input	phishing.input / deepfake.input / log.input	Ommkar
PhishingAgent	cyberguard:phishing.input	threat.detected	Dev 1
DeepfakeAgent	cyberguard:deepfake.input	threat.detected	Dev 2
LogAnalysisAgent	cyberguard:log.input	threat.detected	Dev 3
ThreatScoringAgent	cyberguard:threat.detected	threat.scored	Dev 4
ResponseAgent	cyberguard:threat.scored	threat.complete	Ommkar
Orchestrator DLQ	cyberguard:threat.escalated	threat.complete + MongoDB	Ommkar

§ 05 SOLID Principles

These are enforced at code review. Any PR violating these principles is rejected before functionality is reviewed.

S — Single Responsibility: One class, one job

PhishingAgent routes events only. EmailDetector detects email phishing only. ThreatScoringAgent scores only. Detection and scoring never live in the same class.

O — Open / Closed: Extend without modifying

New ML model = new class implementing BaseMLModel. New LLM = new LLMProvider implementation. New threat type = new agent registered in container.py. Existing classes stay untouched.

L — Liskov Substitution: Any implementation is a valid substitute

Any BaseCyberAgent can be registered with the orchestrator. Any BaseMLModel can replace another inside a detector. Substituting one must not break anything upstream.

I — Interface Segregation: Narrow, focused interfaces

BaseAgent has no scoring methods. BaseScorer has no detection methods. LLMProvider has no awareness of threat types. Never combine responsibilities into one interface.

D — Dependency Inversion: Depend on abstractions always

PhishingAgent receives BaseMLModel in its constructor — not DistilBERT. ResponseAgent receives LLMProvider — not OpenAI. OrchestratorAgent receives a list of BaseCyberAgent — not specific agents. All wiring lives in core/container.py exclusively.

Note: Enforcement rule: if your class imports a concrete implementation directly inside the class body, you violated Dependency Inversion. Move it to the constructor parameter and wire it in container.py.

§ 06 Folder Structure

Backend

```
backend/
├── core/                                # Owner: Ommkar
│   ├── interfaces/
│   │   ├── base_agent.py               # Abstract agent contract
│   │   ├── base_detector.py            # Abstract detector contract
│   │   ├── base_ml_model.py            # Abstract ML model contract
│   │   ├── base_scorer.py              # Abstract scorer contract
│   │   └── llm_provider.py              # Abstract LLM contract
│   ├── events/
│   │   ├── event_bus.py                 # Redis Streams wrapper
│   │   ├── event_types.py               # All Pydantic event schemas
│   │   └── streams.py                   # Stream name constants
│   ├── orchestrator.py                  # AgentScope orchestrator + DLQ handler
│   └── container.py                      # Dependency injection wiring (ONLY file
with concrete imports)
├── agents/
│   ├── phishing/                        # Owner: Dev 1
│   │   ├── agent.py
│   │   ├── models/distilbert_model.py
│   │   └── detectors/
│   │       ├── email_detector.py
│   │       ├── url_detector.py
│   │       ├── sms_detector.py
│   │       └── qr_detector.py
│   ├── deepfake/                        # Owner: Dev 2
│   │   ├── agent.py
│   │   ├── models/efficientnet_model.py
│   │   └── detectors/
│   │       └── image_detector.py, video_detector.py, audio_detector.py
│   ├── log_analysis/                    # Owner: Dev 3
│   │   ├── agent.py
│   │   ├── models/isolation_forest_model.py
│   │   └── detectors/
│   │       └── auth_log_detector.py, system_log_detector.py,
behavior_detector.py
├── api_abuse/                            # Owner: Dev 3 (secondary)
│   ├── agent.py
│   └── detectors/pattern_detector.py
├── scoring/                              # Owner: Dev 4
│   ├── threat_scorer.py                 # Implements BaseScorer
│   └── scoring_agent.py                 # Implements BaseCyberAgent
├── response/                             # Owner: Ommkar
│   ├── response_agent.py
│   └── providers/
│       └── openai_provider.py, groq_provider.py, anthropic_provider.py
├── ingestion/                            # Owner: Dev 4
│   ├── file_upload_handler.py
│   └── redis_stream_handler.py
├── api/                                  # Owner: Dev 4
│   ├── routes/upload.py, stream.py, threats.py, dashboard.py, auth.py
│   └── websocket.py
├── db/                                   # Owner: Ommkar
│   ├── mongodb.py
│   └── repositories/
│       ├── threat_repository.py, session_repository.py
└── main.py, config.py, requirements.txt, .env.example
```

Frontend

```
frontend/                                     # Owner: Dev 5
├── features/
│   ├── phishing/      PhishingUpload.tsx, PhishingResult.tsx, phishing.types.ts
│   ├── deepfake/      DeepfakeUpload.tsx, DeepfakeResult.tsx, deepfake.types.ts
│   ├── logs/          LogStream.tsx, LogEventRow.tsx, log.types.ts
│   └── dashboard/     ThreatStats.tsx, AttackTimeline.tsx, ActionPanel.tsx
├── components/
│   ├── ThreatCard.tsx, RiskBadge.tsx, ExplainPanel.tsx
│   └── MitreBadge.tsx, WebSocketProvider.tsx
├── hooks/
│   └── useThreatStream.ts, useDashboardStats.ts
├── lib/
│   ├── api.ts          # Axios base client + interceptors
│   └── ws.ts           # WebSocket client singleton
├── types/
│   └── threat.ts        # Shared TypeScript types (mirrors backend schemas)
```

§ 07 Core Interfaces

These interfaces are the law of this codebase. Every agent, detector, model, and provider must implement the relevant interface. No exceptions.

BaseCyberAgent — core/interfaces/base_agent.py

```
from abc import ABC, abstractmethod
from agentscope.agents import AgentBase
from agentscope.message import Msg

class BaseCyberAgent(AgentBase, ABC):
    """
    All CyberGuard agents inherit from this.
    RULE: Never call another agent directly inside reply().
    Emit to the event bus. That is the only communication channel.
    """

    @abstractmethod
    def reply(self, x: Msg | None = None) -> Msg:
        """
        AgentScope entry point. Return immediately.
        All heavy work runs inside asyncio.create_task().
        """
        pass

    @abstractmethod
    def subscribes_to(self) -> list[str]:
        """Redis Stream keys this agent consumes."""
        pass

    @abstractmethod
    def emits_to(self) -> list[str]:
        """Redis Stream keys this agent publishes to."""
        pass
```

BaseDetector — core/interfaces/base_detector.py

```
from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from core.interfaces.base_ml_model import BaseMLModel

@dataclass
class DetectionResult:
    label: str # phishing | deepfake | anomaly | safe
    confidence: float # 0.0 to 1.0
    indicators: list[str] = field(default_factory=list) # Human-readable signals
    metadata: dict = field(default_factory=dict) # Raw model output

class BaseDetector(ABC):
    """
    One detector per input modality.
    RULE: Inject BaseMLModel via constructor. Never instantiate inside detect().
    """

    def __init__(self, model: BaseMLModel) -> None:
        self._model = model
```

```

@abstractmethod
async def detect(self, payload: dict) -> DetectionResult:
    """Takes raw payload. Returns DetectionResult. Must be fully async."""
    pass

@abstractmethod
def input_type(self) -> str:
    """Returns: email | url | image | auth_log – etc."""
    pass

```

BaseMLModel — core/interfaces/base_ml_model.py

```

from abc import ABC, abstractmethod

class BaseMLModel(ABC):
    """
    All ML models sit behind this interface.
    Swap DistilBERT for ONNX without touching a single detector.
    """

    @abstractmethod
    async def load(self) -> None:
        """Load weights from disk or HuggingFace hub. Called once at startup."""
        pass

    @abstractmethod
    async def predict(self, input_data: dict) -> dict:
        """Run inference. Detector interprets the output."""
        pass

    @abstractmethod
    def is_loaded(self) -> bool:
        """True if weights are in memory. Used by /health endpoint."""
        pass

    @abstractmethod
    def model_name(self) -> str:
        """Example: distilbert-phishing-v1 – included in explanation metadata."""
        pass

```

LLMProvider — core/interfaces/llm_provider.py

```

from abc import ABC, abstractmethod

class LLMProvider(ABC):
    """
    ResponseAgent depends only on this interface.
    Swap OpenAI to Groq to Anthropic by changing one line in container.py.
    """

    @abstractmethod
    async def explain(self, context: dict) -> str:
        """Generate human-readable threat explanation. Returns one paragraph."""
        pass

    @abstractmethod
    async def recommend(self, threat: dict) -> list[str]:
        """Generate ordered response actions. Highest priority first."""
        pass

```

```
@abstractmethod
async def map_to_mitre(self, threat: dict) -> list[str]:
    """Return MITRE ATT&CK technique IDs. Example: [T1566.001, T1598]"""
    pass
```

BaseScorer — core/interfaces/base_scorer.py

```
from abc import ABC, abstractmethod
from core.events.event_types import ThreatEvent, RiskLevel

class BaseScorer(ABC):
    """
    Separated from detection deliberately – Interface Segregation.
    ThreatScoringAgent injects this. Dev 4 implements ThreatScorer(BaseScorer).
    """

    @abstractmethod
    def score(self, event: ThreatEvent) -> tuple[RiskLevel, float]:
        """Takes detected ThreatEvent. Returns (RiskLevel, confidence)."""
        pass

    @abstractmethod
    def explain_score(self, event: ThreatEvent) -> list[str]:
        """Returns top 3-5 factors that drove this risk level."""
        pass
```

§ 08 Event System

ThreatEvent Schema — core/events/event_types.py

The ThreatEvent is the single data contract flowing through the entire pipeline. Never mutate it. Always use `model_copy(update={...})`.

```
from pydantic import BaseModel, Field
from enum import Enum
from datetime import datetime
import uuid

class ThreatCategory(str, Enum):
    PHISHING = "phishing"
    DEEPFAKE = "deepfake"
    LOG_ANOMALY = "log_anomaly"
    API_ABUSE = "api_abuse"
    UNKNOWN = "unknown"

class InputModality(str, Enum):
    EMAIL = "email" | URL = "url" | SMS = "sms" | QR = "qr"
    IMAGE = "image" | VIDEO = "video" | AUDIO = "audio"
    AUTH_LOG = "auth_log" | SYSTEM_LOG = "system_log" | API_LOG = "api_log"

class RiskLevel(str, Enum):
    SAFE = "safe" | LOW = "low" | MEDIUM = "medium" | HIGH = "high" | CRITICAL = "critical"

class EventStatus(str, Enum):
    RECEIVED = "received" | PROCESSING = "processing" | SCORED = "scored"
    COMPLETE = "complete" | ESCALATED = "escalated" | FAILED = "failed"

class ThreatEvent(BaseModel):
    # Identity
    event_id: str = Field(default_factory=lambda: str(uuid.uuid4()))
    created_at: datetime = Field(default_factory=datetime.utcnow)
    updated_at: datetime = Field(default_factory=datetime.utcnow)

    # Classification
    category: ThreatCategory = ThreatCategory.UNKNOWN
    modality: InputModality
    status: EventStatus = EventStatus.RECEIVED
    source: str # upload | stream
    payload: dict # raw input from ingestion

    # Detection outputs – populated by domain agents
    label: str | None = None
    confidence: float | None = None
    indicators: list[str] = []

    # Scoring outputs – populated by ThreatScoringAgent
    risk_level: RiskLevel | None = None
    score_factors: list[str] = []

    # LLM outputs – populated by ResponseAgent
    explanation: str | None = None
    recommended_actions: list[str] = []
    mitre_mapping: list[str] = []

    # Error tracking – populated on escalation
```

```
error_message: str | None = None
failed_agent: str | None = None
```

Event Bus — core/events/event_bus.py

```
import redis.asyncio as aioredis
from core.events.event_types import ThreatEvent
from config import settings

class EventBus:
    """Agents never import redis directly. They use this class."""

    def __init__(self) -> None:
        self._redis: aioredis.Redis | None = None

    async def connect(self) -> None:
        self._redis = aioredis.from_url(settings.REDIS_URL,
        decode_responses=True)

    async def publish(self, stream: str, event: ThreatEvent) -> str:
        entry_id = await self._redis.xadd(stream, {"data":
event.model_dump_json()})
        return entry_id

    async def consume(self, stream, group, consumer, count=10, block_ms=0):
        entries = await self._redis.xreadgroup(
            groupname=group, consumername=consumer,
            streams={stream: ">"}, count=count, block=block_ms,
        )
        results = []
        for _, messages in entries:
            for entry_id, data in messages:
                event = ThreatEvent.model_validate_json(data["data"])
                results.append((entry_id, event))
        return results

    async def ack(self, stream, group, entry_id) -> None:
        await self._redis.xack(stream, group, entry_id)

    async def create_consumer_group(self, stream, group) -> None:
        try:
            await self._redis.xgroup_create(stream, group, id="0", mkstream=True)
        except aioredis.ResponseError:
            pass # Group already exists – safe to ignore

event_bus = EventBus() # Singleton – inject via container.py
```

Stream Names — Canonical Reference

Warning: Never hardcode stream names inline. Import from core/events/streams.py. These names are the single source of truth.

STREAM KEY	PRODUCER	CONSUMER
cyberguard:raw.input	Ingestion Layer	OrchestratorAgent
cyberguard:phishing.input	OrchestratorAgent	PhishingAgent
cyberguard:deepfake.input	OrchestratorAgent	DeepfakeAgent
cyberguard:log.input	OrchestratorAgent	LogAnalysisAgent

STREAM KEY	PRODUCER	CONSUMER
cyberguard:threat.detected	Domain Agents (1, 2, 3)	ThreatScoringAgent
cyberguard:threat.scored	ThreatScoringAgent	ResponseAgent
cyberguard:threat.complete	ResponseAgent + DLQ handler	WebSocket broadcast layer
cyberguard:threat.escalated	Any failed agent	OrchestratorAgent DLQ handler

§ 09 Agent Contracts

OrchestratorAgent — core/orchestrator.py (Owner: Ommkar)

```
import asyncio
from agentscope.message import Msg
from core.interfaces.base_agent import BaseCyberAgent
from core.events.event_bus import EventBus
from core.events.event_types import ThreatEvent, InputModality, EventStatus

MODALITY_ROUTES: dict[InputModality, str] = {
    InputModality.EMAIL: "cyberguard:phishing.input",
    InputModality.URL: "cyberguard:phishing.input",
    InputModality.SMS: "cyberguard:phishing.input",
    InputModality.QR: "cyberguard:phishing.input",
    InputModality.IMAGE: "cyberguard:deepfake.input",
    InputModality.VIDEO: "cyberguard:deepfake.input",
    InputModality.AUDIO: "cyberguard:deepfake.input",
    InputModality.AUTH_LOG: "cyberguard:log.input",
    InputModality.SYSTEM_LOG: "cyberguard:log.input",
    InputModality.API_LOG: "cyberguard:log.input",
}

class OrchestratorAgent(BaseCyberAgent):

    def __init__(self, name="orchestrator", bus: EventBus = None) -> None:
        super().__init__(name=name)
        self._bus = bus

    def subscribes_to(self) -> list[str]:
        return ["cyberguard:raw.input", "cyberguard:threat.escalated"]

    def emits_to(self) -> list[str]:
        return ["cyberguard:phishing.input", "cyberguard:deepfake.input",
"cyberguard:log.input"]

    def reply(self, x: Msg | None = None) -> Msg:
        event = ThreatEvent.model_validate(x.content)
        asyncio.create_task(self._route(event))
        return Msg(name=self.name, content={"status": "routing"},
role="assistant")

    async def _route(self, event: ThreatEvent) -> None:
        target = MODALITY_ROUTES.get(event.modality)
        if not target:
            await self._bus.publish("cyberguard:threat.escalated",
event.model_copy(update={
                "status": EventStatus.ESCALATED,
                "error_message": f"No route for modality: {event.modality}",
                "failed_agent": self.name,
            }))
            return
        await self._bus.publish(target, event)

    async def handle_escalation(self, event: ThreatEvent) -> None:
        """Dead Letter Queue — called when consuming
cyberguard:threat.escalated"""
        await self._repo.save_escalation(event)
        await self._bus.publish("cyberguard:threat.complete", event)
```

PhishingAgent — agents/phishing/agent.py (Owner: Dev 1)

```
import asyncio
from agentscope.message import Msg
from core.interfaces.base_agent import BaseCyberAgent
from core.interfaces.base_detector import BaseDetector
from core.events.event_bus import EventBus
from core.events.event_types import ThreatEvent, InputModality, EventStatus

class PhishingAgent(BaseCyberAgent):
    """Routes phishing inputs to correct detector by modality.
    Contains NO detection logic. Detection lives in detectors/."""

    def __init__(self, name="phishing_agent", bus: EventBus = None,
                 detectors: dict[InputModality, BaseDetector] = None) -> None:
        super().__init__(name=name)
        self._bus = bus
        self._detectors = detectors or {}

    def subscribes_to(self) -> list[str]:
        return ["cyberguard:phishing.input"]

    def emits_to(self) -> list[str]:
        return ["cyberguard:threat.detected", "cyberguard:threat.escalated"]

    def reply(self, x: Msg | None = None) -> Msg:
        event = ThreatEvent.model_validate(x.content)
        asyncio.create_task(self._detect(event))
        return Msg(name=self.name, content={"status": "detecting"},
                  role="assistant")

    async def _detect(self, event: ThreatEvent) -> None:
        try:
            detector = self._detectors.get(event.modality)
            if not detector:
                raise ValueError(f"No detector for modality: {event.modality}")
            result = await detector.detect(event.payload)
            await self._bus.publish("cyberguard:threat.detected",
                                   event.model_copy(update={
                                       "label": result.label, "confidence": result.confidence,
                                       "indicators": result.indicators, "status":
EventStatus.PROCESSING,
                                   }))
        except Exception as exc:
            await self._bus.publish("cyberguard:threat.escalated",
                                   event.model_copy(update={
                                       "status": EventStatus.ESCALATED,
                                       "error_message": str(exc), "failed_agent": self.name,
                                   }))
```

Note: Dev 2 and Dev 3: DeepfakeAgent and LogAnalysisAgent follow the exact same pattern as PhishingAgent. Copy this file, rename the class, change subscribes_to() to the correct stream, and inject your detector map. All detection logic lives in your detectors, not here.

ThreatScoringAgent — scoring/scoring_agent.py (Owner: Dev 4)

```
import asyncio
from agentscope.message import Msg
from core.interfaces.base_agent import BaseCyberAgent
from core.interfaces.base_scorer import BaseScorer
from core.events.event_bus import EventBus
```

```

from core.events.event_types import ThreatEvent, EventStatus

class ThreatScoringAgent(BaseCyberAgent):
    def __init__(self, name="threat_scoring_agent", bus=None, scorer: BaseScorer = None):
        super().__init__(name=name)
        self._bus = bus
        self._scorer = scorer

    def subscribes_to(self): return ["cyberguard:threat.detected"]
    def emits_to(self): return ["cyberguard:threat.scored",
                                "cyberguard:threat.escalated"]

    def reply(self, x=None) -> Msg:
        event = ThreatEvent.model_validate(x.content)
        asyncio.create_task(self._score(event))
        return Msg(name=self.name, content={"status": "scoring"},
                    role="assistant")

    async def _score(self, event: ThreatEvent) -> None:
        try:
            risk_level, confidence = self._scorer.score(event)
            score_factors = self._scorer.explain_score(event)
            await self._bus.publish("cyberguard:threat.scored",
                                    event.model_copy(update={
                                        "risk_level": risk_level, "confidence": confidence,
                                        "score_factors": score_factors, "status": EventStatus.SCORED,
                                    }))
        except Exception as exc:
            await self._bus.publish("cyberguard:threat.escalated",
                                    event.model_copy(update={
                                        "status": EventStatus.ESCALATED,
                                        "error_message": str(exc), "failed_agent": self.name,
                                    }))

```

ResponseAgent — response/response_agent.py (Owner: Ommkar)

```

import asyncio
from agentscope.message import Msg
from core.interfaces.base_agent import BaseCyberAgent
from core.interfaces.llm_provider import LLMPProvider
from core.events.event_bus import EventBus
from core.events.event_types import ThreatEvent, EventStatus

class ResponseAgent(BaseCyberAgent):
    """Final agent in the pipeline.
    Calls LLMPProvider for explanation, recommendations, MITRE mapping.
    Persists complete ThreatEvent to MongoDB.
    Broadcasts to threat.complete for WebSocket delivery."""

    def __init__(self, name="response_agent", bus=None,
                 llm: LLMPProvider = None, repository=None):
        super().__init__(name=name)
        self._bus = bus; self._llm = llm; self._repo = repository

    def subscribes_to(self): return ["cyberguard:threat.scored"]
    def emits_to(self): return ["cyberguard:threat.complete",
                                "cyberguard:threat.escalated"]

    def reply(self, x=None) -> Msg:
        event = ThreatEvent.model_validate(x.content)

```

```

        asyncio.create_task(self._respond(event))
        return Msg(name=self.name, content={"status": "generating_response"},
role="assistant")

    async def _respond(self, event: ThreatEvent) -> None:
        try:
            ctx = {
                "category": event.category, "modality": event.modality,
                "indicators": event.indicators, "confidence": event.confidence,
                "risk_level": event.risk_level, "score_factors":
event.score_factors,
            }
            explanation = await self._llm.explain(ctx)
            actions      = await self._llm.recommend(ctx)
            mitre         = await self._llm.map_to_mitre(ctx)
            complete = event.model_copy(update={
                "explanation": explanation, "recommended_actions": actions,
                "mitre_mapping": mitre, "status": EventStatus.COMPLETE,
            })
            await self._repo.save(complete)
            await self._bus.publish("cyberguard:threat.complete", complete)
        except Exception as exc:
            await self._bus.publish("cyberguard:threat.escalated",
event.model_copy(update={
                "status": EventStatus.ESCALATED,
                "error_message": str(exc), "failed_agent": self.name,
            }))

```

§ 10 MongoDB Schema

Owner: Ommkar. No other developer modifies collection schemas without review and an update to this section.

COLLECTION	PURPOSE	TTL
threats	All complete ThreatEvents — primary query target	None (permanent)
escalations	Failed events requiring manual review	7 days (604800 seconds)
sessions	JWT session tracking	24 hours

threats — Document Shape

```
{
  "_id": "ObjectId",
  "event_id": "uuid — primary query key",
  "created_at": "ISODate",
  "updated_at": "ISODate",
  "category": "phishing | deepfake | log_anomaly | api_abuse",
  "modality": "email | url | sms | qr | image | video | audio | auth_log | system_log | api_log",
  "status": "complete | escalated",
  "source": "upload | stream",
  "risk_level": "safe | low | medium | high | critical",
  "confidence": 0.94,
  "label": "phishing",
  "indicators": ["Sender domain spoofing", "Urgent credential request"],
  "score_factors": ["Confidence above HIGH threshold (0.94)"],
  "explanation": "High Risk: The sender domain closely resembles an authorised...",
  "recommended_actions": ["Quarantine email", "Warn user", "Block sender domain"],
  "mitre_mapping": ["T1566.001", "T1598"],
  "error_message": null,
  "failed_agent": null,
  "payload": {}
}
```

Required Indexes — run at startup in db/mongodb.py

```
async def create_indexes(db) -> None:
    await db.threats.create_index("event_id", unique=True)
    await db.threats.create_index([("created_at", -1)])
    await db.threats.create_index("risk_level")
    await db.threats.create_index("category")
    await db.threats.create_index("status")
    await db.threats.create_index([("risk_level", 1), ("created_at", -1)])

    # TTL — auto-delete after specified seconds
    await db.sessions.create_index("expires_at", expireAfterSeconds=0)
    await db.escalations.create_index("created_at", expireAfterSeconds=604800)
```

§ 11 FastAPI Routes

Owner: Dev 4. All routes require JWT authentication except /auth/login and /health.

METH OD	ENDPOINT	AUTH	PURPOSE
POST	/api/v1/auth/login	None	Issue JWT token
POST	/api/v1/upload	JWT	Upload file — .eml, image, video, audio, .csv log
POST	/api/v1/stream/ingest	JWT	Push raw event JSON to Redis Stream directly
GET	/api/v1/threats	JWT	Paginated list — filters: risk_level, category, date
GET	/api/v1/threats/{event_id}	JWT	Single threat with full explanation and MITRE mapping
GET	/api/v1/dashboard/stats	JWT	Aggregated counts, timeline, and top targets
GET	/api/v1/health	None	Redis, MongoDB, and model load status
WS	/ws/threats	JWT (query param)	Real-time threat stream to dashboard

Dashboard Stats Response Shape

```
{
  "total_events":      1847,
  "threats_detected":  234,
  "safe_events":       1613,
  "by_category":       { "phishing": 145, "deepfake": 61, "log_anomaly": 28 },
  "by_risk_level":     { "critical": 12, "high": 47, "medium": 89, "low": 86 },
  "attack_timeline": [
    { "timestamp": "2024-10-14T10:00:00Z", "count": 14 },
    { "timestamp": "2024-10-14T11:00:00Z", "count": 31 }
  ],
  "frequently_targeted": ["admin@org.com", "finance-portal", "login-page"],
  "recent_threats":     [ "...last 5 complete ThreatEvents..." ]
}
```

§ 12 JWT Authentication

api/routes/auth.py — Owner: Dev 4

```
from fastapi import APIRouter, Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
from jose import JWTError, jwt
from datetime import datetime, timedelta
from config import settings

router = APIRouter(prefix="/api/v1/auth")
oauth2 = OAuth2PasswordBearer(tokenUrl="/api/v1/auth/login")
ALGORITHM = "HS256"
EXPIRE_M = 60 * 24 # 24-hour tokens

def create_access_token(data: dict) -> str:
    to_encode = {"**data", "exp": datetime.utcnow() + timedelta(minutes=EXPIRE_M)}
    return jwt.encode(to_encode, settings.JWT_SECRET, algorithm=ALGORITHM)

async def get_current_user(token: str = Depends(oauth2)) -> dict:
    try:
        return jwt.decode(token, settings.JWT_SECRET, algorithms=[ALGORITHM])
    except JWTError:
        raise HTTPException(status_code=401, detail="Invalid or expired token",
                             headers={"WWW-Authenticate": "Bearer"})

@router.post("/login")
async def login(form: OAuth2PasswordRequestForm = Depends()):
    if form.username != settings.ADMIN_USER or form.password != settings.ADMIN_PASS:
        raise HTTPException(status_code=400, detail="Invalid credentials")
    token = create_access_token({"sub": form.username, "role": "admin"})
    return {"access_token": token, "token_type": "bearer"}

# Protecting a route:
# @router.get("/api/v1/threats")
# async def get_threats(user: dict = Depends(get_current_user)):

# WebSocket auth — client connects with token as query param:
# ws://host:8000/ws/threats?token=eyJ...
```

§ 13 Error Handling

When any agent fails, it escalates. It does not retry. It does not log silently. It publishes to `cyberguard:threat.escalated` and the `OrchestratorAgent` DLQ handler persists and broadcasts it.

Escalation Flow

Agent Exception → Publish to escalated stream → Orchestrator DLQ → Save to escalations collection → Broadcast via WebSocket → Dashboard red alert

Mandatory Pattern — Every Agent

```
async def _process(self, event: ThreatEvent) -> None:
    try:
        # ... agent logic ...
        pass
    except Exception as exc:
        await self._bus.publish(
            "cyberguard:threat.escalated",
            event.model_copy(update={
                "status":      EventStatus.ESCALATED,
                "error_message": str(exc),
                "failed_agent": self.name,
            })
        )
```

Critical: Every except block must publish to the escalated stream. Silent failures are banned. An exception that disappears into a log file causes a missing event in the dashboard that nobody investigates until demo day.

§ 14 WebSocket Protocol

Owner: Dev 4 + Dev 5 jointly. This schema must be finalized and signed off before either side builds their WebSocket implementation.

Critical: Unfinalized as of doc v1.0.0. Dev 4 and Dev 5 must agree, update this section, and both sign off before implementation begins.

Proposed Message Format (Pending Sign-off)

```
{
  "type": "threat.complete",
  "event_id": "uuid",
  "timestamp": "ISO8601",
  "category": "phishing",
  "modality": "email",
  "risk_level": "high",
  "confidence": 0.94,
  "label": "phishing",
  "indicators": ["Domain spoofing", "Urgent credential request", "Malicious redirect"],
  "explanation": "High Risk: The sender domain closely resembles...",
  "recommended_actions": ["Quarantine email", "Warn user", "Block sender domain"],
  "mitre_mapping": ["T1566.001", "T1598"]
}
```

MESSAGE TYPE	DIRECTION	TRIGGER
threat.complete	Server → Client	Threat fully processed and persisted
threat.escalated	Server → Client	Agent failed — manual review needed
stats.update	Server → Client	Dashboard counters updated after each event
ping	Client → Server	Keep-alive every 30 seconds
pong	Server → Client	Keep-alive response

§ 15 Frontend Component Architecture

Owner: Dev 5. One unified dashboard. Feature panels for phishing, deepfake, and logs. Shared component library.

Shared TypeScript Types — types/threat.ts

```
export type ThreatCategory = "phishing" | "deepfake" | "log_anomaly" |
"api_abuse"
export type RiskLevel      = "safe" | "low" | "medium" | "high" | "critical"
export type EventStatus    = "received" | "processing" | "scored" | "complete" |
"escalated"

export interface ThreatEvent {
  event_id:      string
  created_at:    string
  category:      ThreatCategory
  modality:      string
  status:        EventStatus
  risk_level:    RiskLevel | null
  confidence:    number | null
  label:         string | null
  indicators:    string[]
  explanation:   string | null
  recommended_actions: string[]
  mitre_mapping: string[]
  error_message: string | null
}

export interface DashboardStats {
  total_events:      number
  threats_detected:  number
  by_category:       Record<ThreatCategory, number>
  by_risk_level:      Record<RiskLevel, number>
  attack_timeline:   Array<{ timestamp: string; count: number }>
  frequently_targeted: string[]
}
```

useThreatStream — hooks/useThreatStream.ts

```
import { useEffect, useState, useRef } from "react"
import type { ThreatEvent } from "@types/threat"

export function useThreatStream(token: string) {
  const [threats, setThreats] = useState<ThreatEvent[]>([])
  const [connected, setConnected] = useState(false)
  const wsRef = useRef<WebSocket | null>(null)

  useEffect(() => {
    const ws = new WebSocket(
      `${process.env.NEXT_PUBLIC_WS_URL}/ws/threats?token=${token}`
    )
    wsRef.current = ws
    ws.onopen = () => setConnected(true)
    ws.onclose = () => setConnected(false)
    ws.onmessage = (e) => {
      const msg = JSON.parse(e.data as string)
      if (msg.type === "threat.complete" || msg.type === "threat.escalated") {
```

```

        setThreats(prev => [msg.data, ...prev].slice(0, 100))
      }
    }
    const ping = setInterval(() => {
      if (ws.readyState === WebSocket.OPEN)
        ws.send(JSON.stringify({ type: "ping" }))
    }, 30_000)
    return () => { clearInterval(ping); ws.close() }
  }, [token])

  return { threats, connected }
}

```

RiskBadge — components/RiskBadge.tsx

```

import type { RiskLevel } from "@types/threat"

const STYLES: Record<RiskLevel, string> = {
  safe: "bg-emerald-500/10 text-emerald-400 border-emerald-500/25",
  low: "bg-cyan-500/10 text-cyan-400 border-cyan-500/25",
  medium: "bg-amber-500/10 text-amber-400 border-amber-500/25",
  high: "bg-orange-500/10 text-orange-400 border-orange-500/25",
  critical: "bg-red-500/10 text-red-400 border-red-500/25",
}

export function RiskBadge({ level }: { level: RiskLevel }) {
  return (
    <span className={`inline-flex items-center gap-1.5 px-2.5 py-1
      rounded border text-xs font-mono font-semibold uppercase ${STYLES[level]}`}
    >
      <span className="w-1.5 h-1.5 rounded-full bg-current" />
      {level}
    </span>
  )
}

```

§ 16 Datasets

Critical: Dataset pipeline owner not assigned. Assign before Day 2.

DATASET	AGENT	SOURCE	FORMAT
PhishTank	PhishingAgent (URL)	phishtank.org/developer_info.php	CSV, daily updated
IWSPA-2018	PhishingAgent (email)	iwspa.net/shared/	Text + labels
SMS Spam Collection	PhishingAgent (SMS)	UCI ML Repository	TSV
FaceForensics++	DeepfakeAgent (image/video)	github.com/ondyari/FaceForensics	MP4 frames + labels
ASVspoof 2019	DeepfakeAgent (audio)	datashare.ed.ac.uk	WAV + labels
CERT Insider Threat v6.2	LogAnalysisAgent	resources.sei.cmu.edu/library	CSV logs
UNSW-NB15	APIAbuseAgent	research.unsw.edu.au/projects/unsw-nb15	CSV

§ 17 Dev Setup

Prerequisites

- Python 3.11+
- Node.js 20+
- Docker + Docker Compose
- Git

1 — Clone and Configure

```
git clone https://github.com/your-org/cyberguard
cd cyberguard
cp backend/.env.example backend/.env
# Edit backend/.env with your values
```

2 — Environment Variables (backend/.env)

```
MONGODB_URL=mongodb://localhost:27017
MONGODB_DB=cyberguard
REDIS_URL=redis://localhost:6379
JWT_SECRET=replace-with-a-random-64-character-string
ADMIN_USER=admin
ADMIN_PASS=cyberguard2024

# LLM — fill whichever you are using
OPENAI_API_KEY=
GROQ_API_KEY=
ANTHROPIC_API_KEY=
LLM_PROVIDER=groq          # openai | groq | anthropic
```

3 — Start Infrastructure

```
docker compose up -d mongodb redis
```

4 — Start Backend

```
cd backend
python -m venv venv
source venv/bin/activate    # Windows: venv\Scripts\activate
pip install -r requirements.txt
uvicorn main:app --reload --port 8000
```

5 — Start Frontend

```
cd frontend
npm install
cp .env.local.example .env.local
```

```
# NEXT_PUBLIC_API_URL=http://localhost:8000
# NEXT_PUBLIC_WS_URL=ws://localhost:8000
npm run dev
```

requirements.txt (minimum)

```
fastapi>=0.111.0
uvicorn[standard]>=0.30.0
agentscope==1.0.21
pydantic>=2.0.0
pydantic-settings>=2.0.0
redis[hiredis]>=5.0.0
motor>=3.4.0
python-jose[cryptography]>=3.3.0
python-multipart>=0.0.9
transformers>=4.40.0
torch>=2.0.0
scikit-learn>=1.4.0
Pillow>=10.0.0
librosa>=0.10.0
openai>=1.30.0
anthropic>=0.28.0
```

§ 18 Git Workflow

BRANCH	PURPOSE	MERGES TO
main	Demo-ready code only — never broken	—
dev	Integration branch — merged to main on demo day only	main
feat/phishing-agent	Dev 1 feature work	dev
feat/deepfake-agent	Dev 2 feature work	dev
feat/log-analysis	Dev 3 feature work	dev
feat/fastapi	Dev 4 feature work	dev
feat/dashboard	Dev 5 feature work	dev
fix/[bug-name]	Bug fixes	dev

Commit Convention

```
feat(phishing): add email detector with DistilBERT model
fix(orchestrator): handle unknown modality with escalation
docs(api): update dashboard stats response schema
refactor(scoring): extract risk thresholds to config
test(deepfake): add image detector unit tests
chore(deps): pin agentscope to 1.0.21
```

Warning: Never push directly to main or dev. Always open a pull request. Minimum one reviewer. Module owner must approve any PR touching their module.

§ 19 Coding Standards

Non-Negotiable Rules

1. Type hints on every function and method. No `def foo(x)`. Always `def foo(x: dict) -> str`.
2. No concrete imports inside class bodies. All wiring lives in `core/container.py` exclusively.
3. No direct Redis or MongoDB calls inside agents or detectors. Use injected EventBus and repository objects.
4. Async throughout. Every I/O operation uses `async/await`. No `time.sleep()`. No blocking calls in async functions.
5. Pydantic for all data models. No raw dicts crossing module boundaries. Validate at ingestion boundary.
6. Every agent escalates on exception. No silent failures. No logging-only error handlers.
7. One class per file. Exceptions: trivially related dataclasses only.
8. Frontend TypeScript strict mode. No any types anywhere in frontend/.
9. Never mutate a ThreatEvent. Always use `model_copy(update={...})`.
10. Stream name constants only. Never hardcode stream names inline. Import from `core/events/streams.py`.

Naming Conventions

TARGET	CONVENTION	EXAMPLE
Python files	snake_case	<code>email_detector.py</code>
Python classes	PascalCase	<code>class EmailDetector</code>
Python async functions	snake_case	<code>async def detect()</code>
Python constants	SCREAMING_SNAKE	<code>MODALITY_ROUTES</code>
TypeScript components	PascalCase	<code>ThreatCard.tsx</code>
TypeScript hooks	camelCase with use prefix	<code>useThreatStream.ts</code>
TypeScript interfaces	PascalCase	<code>interface ThreatEvent</code>
Redis stream keys	cyberguard:noun.verb	<code>cyberguard:threat.complete</code>
MongoDB collections	lowercase plural	<code>threats, sessions</code>
REST endpoints	kebab-case	<code>/api/v1/dashboard/stats</code>

Risk Level Thresholds — Dev 4 implements in ThreatScorer

RISK LEVEL	CONFIDENCE RANGE	MEANING
SAFE	0.00 — 0.30	No threat indicators — clean
LOW	0.30 — 0.55	Suspicious but inconclusive — log and monitor
MEDIUM	0.55 — 0.75	Probable threat — warn user
HIGH	0.75 — 0.90	Likely threat — quarantine and notify
CRITICAL	0.90 — 1.00	Confirmed threat — block, escalate, notify SOC